

Rapport TP Data Lake — PokemonLake

Partie A — Mise en place du stockage objet avec MinIO

Ajout de MinIO dans l'environnement Docker

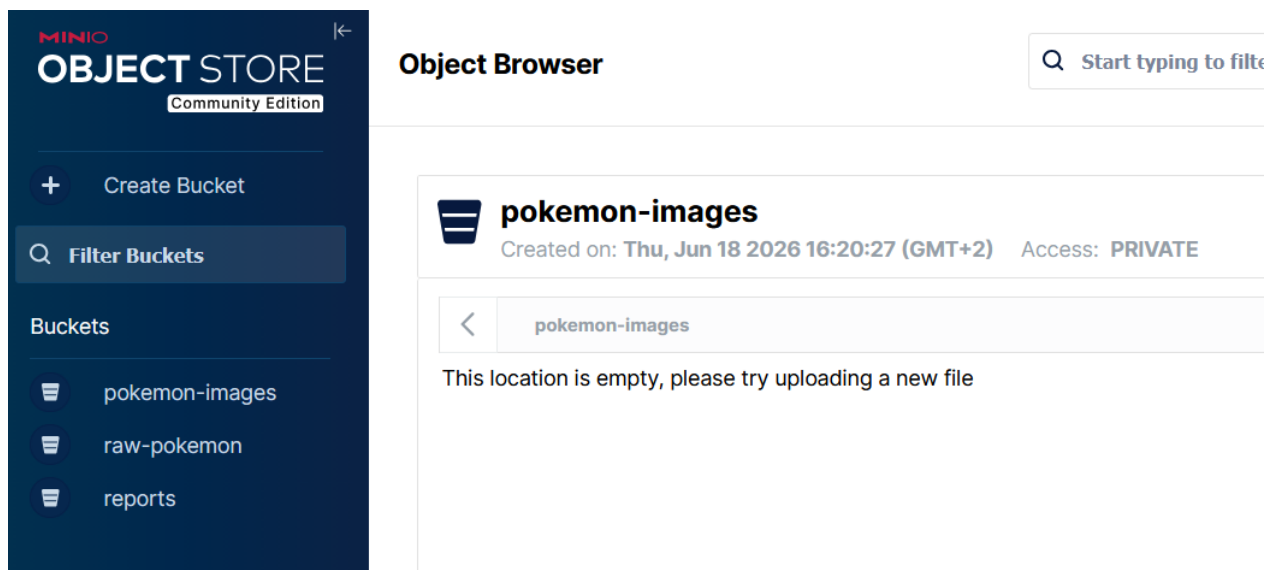
MinIO a été ajouté en tant que service dans le fichier `docker-compose.yml`.
Il expose deux ports : **9002** pour l'API S3 et **9003** pour la console web.

Un conteneur d'initialisation (`minio-pokemon-init`) crée automatiquement les trois buckets au premier démarrage.

Organisation logique retenue : plusieurs buckets distincts

Bucket	Contenu
raw-pokemon	Réponses JSON brutes issues de la PokéAPI
pokemon-images	Images officielles et sprites des Pokémon
reports	Rapports CSV ou JSON générés

Le choix de plusieurs buckets distincts — plutôt qu'un bucket unique avec préfixes — permet de gérer des politiques d'accès différenciées par type de données et de séparer clairement les responsabilités entre données brutes, médias et rapports.



Partie B — Base enrichie

Tables créées

Deux tables ont été ajoutées dans la base `pokemon_lake` (PostgreSQL, port 5434).

```
PS C:\Users\lilas\Desktop\DataLake\PokemonLake> docker exec -it postgres-pokemon psql -U pokemon_user -d pokemon_lake -c "\dt"
List of relations
Schema |      Name      | Type | Owner
-----+-----+-----+-----
public | file_ingestion_log | table | pokemon_user
public | pokemon_files    | table | pokemon_user
(2 rows)
```

Table `pokemon_files`

Catalogue des fichiers stockés dans MinIO, liés à un Pokémon. La base ne stocke pas le fichier lui-même mais uniquement ses métadonnées et un pointeur (`object_key`) vers l'objet dans le bucket.

Colonne	Type	Description
<code>file_id</code>	SERIAL PK	Identifiant unique
<code>pokemon_id</code>	INTEGER	ID du Pokémon (PokéAPI)
<code>bucket_name</code>	VARCHAR	Nom du bucket MinIO
<code>object_key</code>	VARCHAR	Chemin de l'objet dans le bucket
<code>file_name</code>	VARCHAR	Nom du fichier
<code>file_type</code>	VARCHAR	Extension (json, png, csv...)
<code>created_at</code>	TIMESTAMPTZ	Date de référencement
<code>file_size_bytes</code>	BIGINT	Taille en octets
<code>mime_type</code>	VARCHAR	Type MIME
<code>internal_url</code>	TEXT	URL S3 interne
<code>checksum</code>	VARCHAR	SHA-256 pour vérification d'intégrité

Table `file_ingestion_log`

Journal de toutes les tentatives d'ingestion, succès et erreurs inclus.

Colonne	Type	Description
<code>log_id</code>	SERIAL PK	Identifiant unique
<code>file_name</code>	VARCHAR	Nom du fichier ingéré
<code>bucket_name</code>	VARCHAR	Bucket cible
<code>object_key</code>	VARCHAR	Chemin dans le bucket
<code>processed_at</code>	TIMESTAMPTZ	Date de traitement
<code>source</code>	VARCHAR	Origine (pokeapi, manual...)

Colonne	Type	Description
status	VARCHAR	success, error ou pending
error_message	TEXT	Message d'erreur si échec
file_size_bytes	BIGINT	Taille ingérée

Partie C — Workflow n8n

Description du workflow

Le workflow orchestre l'ingestion complète du JSON brut de Bulbizarre (Pokémon #1) depuis la PokéAPI vers MinIO et PostgreSQL.

Déclenchement Manuel

PokéAPI — GET /pokemon/bulbasaur

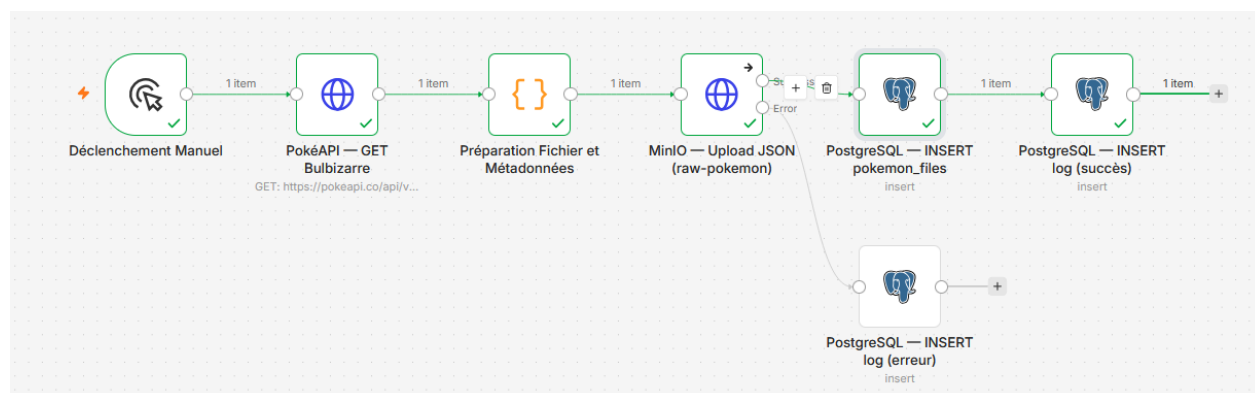
Préparation Fichier et Métadonnées

MinIO — Upload JSON (bucket raw-pokemon)

(succès) → PostgreSQL — INSERT pokemon_files

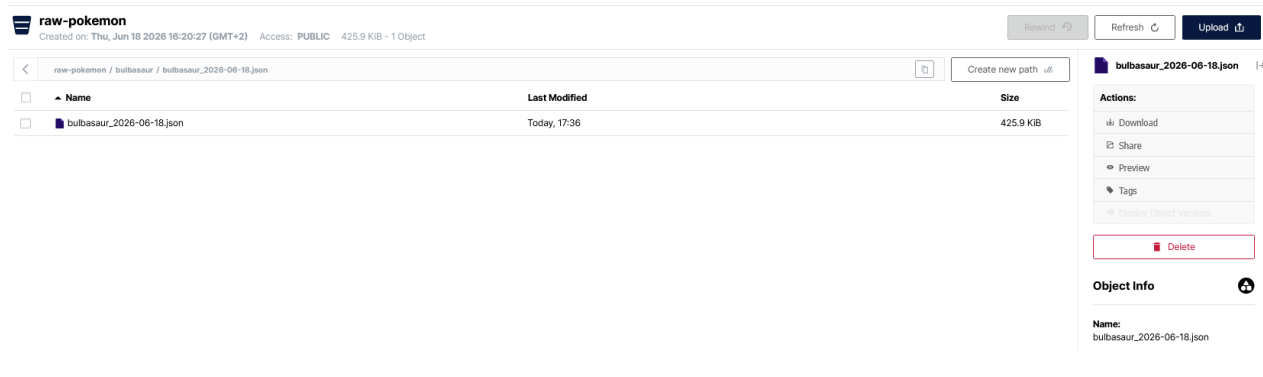
PostgreSQL — INSERT log (succès)

(erreur) → PostgreSQL — INSERT log (erreur)



Objet stocké dans MinIO

Le fichier `bulbasaur_2026-06-18.json` (425.9 KiB) a été déposé dans le bucket `raw-pokemon` sous le chemin `bulbasaur/bulbasaur_2026-06-18.json`. Il contient la réponse brute complète de la PokéAPI, sans transformation.



Partie D — Pourquoi cette architecture est un Data Lake / Lakehouse

L'architecture obtenue dépasse la logique d'une simple base relationnelle car elle repose sur une séparation explicite entre le stockage des fichiers et le stockage des métadonnées. MinIO apporte une vraie couche de stockage objet complémentaire : il conserve les fichiers bruts dans leur format d'origine (JSON, PNG, CSV), sans transformation, ce qui permet de les rejouer ou de les retraiter à tout moment sans dépendre à nouveau d'une API externe. La base PostgreSQL, elle, ne contient pas les fichiers : elle contient uniquement leurs métadonnées et un pointeur (`object_key`) vers l'objet dans le bucket. Ce découplage est fondamental : stocker des fichiers binaires en base relationnelle serait coûteux, non scalable, et inadapté à des formats hétérogènes. La table `file_ingestion_log` ajoute une traçabilité complète de tous les flux entrants (source, statut, date, erreur éventuelle), ce qui rapproche le projet d'une logique Lakehouse : on sait quoi a été ingéré, quand, et avec quel résultat. Une simple base relationnelle ne pourrait offrir ni le stockage fichier à faible coût, ni cette séparation des responsabilités entre brut et structuré.